TÀI LIỆU KỸ THUẬT

RAG Từ Cơ Bản Đến Thực Chiến

Xây Hệ Thống AI Biết Tra Cứu Dữ Liệu

Lộ trình thực chiến dành cho kỹ sư phần mềm: từ lúc có dữ liệu lộn xộn cho đến khi triển khai hệ thống AI tra cứu chính xác, tối ưu chi phí và độ trễ ở quy mô lớn.

Mục lục

Mở đầu

Phần 1: RAG là gì và vì sao cần RAG

Chương 1: LLM giới hạn ở đâu và vì sao cần RAG

1.1 Ảo giác và giới hạn ngữ cảnh dưới góc nhìn Developer

1.2 So sánh RAG và Fine-tuning cho ứng dụng phần mềm

Chương 2: RAG hoạt động như thế nào (high-level pipeline)

2.1 Luồng đi của dữ liệu: Từ API Request đến Response

2.2 Kiến trúc Naive RAG vs Advanced RAG cho Web App

Chương 3: Các thành phần chính: embedding, vector DB, retriever, generator

3.1 Retriever và Generator: Interface và sự tương tác

3.2 Vai trò của Embedding và Vector Database trong Stack công nghệ

Phần 2: Xây nền tảng dữ liệu cho RAG

Chương 4: Embedding là gì và cách chọn Model phù hợp

4.1 Cơ chế hoạt động của Embedding dưới góc nhìn lập trình

4.2 Tiêu chí chọn Embedding API vs Self-hosted

Chương 5: Chunking: Chiến lược phân mảnh tài liệu

5.1 Các thuật toán Chunking phổ biến cho Developer

5.2 Tối ưu Chunk Size và Overlap cho ứng dụng thực tế

Chương 6: Vector Database: Lựa chọn và tích hợp

6.1 Kiến trúc và cách hoạt động của Vector DB

6.2 So sánh FAISS, Pinecone và Weaviate cho Software Project

Chương 7: Xây dựng Pipeline Indexing tự động

7.1 Thiết kế luồng ETL cho dữ liệu Vector

7.2 Quản lý Metadata và Filtering trong ứng dụng

Phần 3: Tối ưu hóa Retrieval và chất lượng phản hồi

Chương 8: Similarity Search và các phép đo khoảng cách

8.1 Cosine Similarity và Euclidean Distance trong thực tế

8.2 Thuật toán ANN: Tăng tốc độ tìm kiếm cho ứng dụng lớn

Chương 9: Hybrid Search: Kết hợp Keyword và Vector

9.1 Tại sao Vector Search thuần túy là chưa đủ?

9.2 Triển khai Hybrid Search với BM25 và Vector

Chương 10: Reranking: Kỹ thuật lọc kết quả chính xác

10.1 Cơ chế Cross-Encoders trong Pipeline RAG
10.2 Cân bằng giữa Accuracy và API Latency
Chương 11: Context Management và Prompt Engineering
11.1 Chiến thuật sắp xếp ngữ cảnh trong Prompt
11.2 Xây dựng Guardrails cho câu trả lời của AI

Phần 4: Kiến trúc RAG nâng cao cho ứng dụng phức tạp

Chương 12: Multi-hop RAG: Truy xuất đa luồng
12.1 Kỹ thuật Query Decomposition trong Code
12.2 Tổng hợp kết quả từ nhiều bước truy xuất
Chương 13: Agentic RAG: AI tự động hóa quy trình
13.1 Vòng lặp ReAct (Reason + Act) cho RAG
13.2 Xây dựng Tool-set cho RAG Agent
Chương 14: Quản lý Memory và Long-term Context
14.1 Thiết kế Session và Chat History Store
14.2 Kỹ thuật tóm tắt ngữ cảnh để tiết kiệm Token
Chương 15: Kết hợp RAG với SQL và Graph Database
15.1 Text-to-SQL: Truy vấn DB an toàn
15.2 GraphRAG: Tận dụng mối quan hệ giữa các thực thể

Phần 5: Triển khai thực tế và vận hành ứng dụng

Chương 16: Xây dựng Chatbot tài liệu nội bộ
16.1 Kiến trúc Full-stack cho Chatbot RAG
16.2 Xử lý đa định dạng file: PDF, Markdown, Wiki
Chương 17: Nâng cấp Search Engine với RAG
17.1 Tích hợp RAG vào luồng tìm kiếm hiện có
17.2 Trích dẫn nguồn và tính minh bạch của dữ liệu
Chương 18: Phát triển API RAG với FastAPI và Next.js
18.1 Xây dựng Backend API hỗ trợ Streaming
18.2 Frontend Integration và xử lý Stream dữ liệu
Chương 19: Scaling, Latency và Tối ưu hóa chi phí
19.1 Kỹ thuật Caching cho ứng dụng RAG
19.2 Monitoring và Quản lý chi phí Token
Chương 20: Những lỗi phổ biến và Checklist Production
20.1 Top 5 sai lầm kiến trúc khiến dự án RAG thất bại
20.2 Checklist sẵn sàng cho Production

Mở đầu

Bạn đang đọc những dòng này hẳn đã từng ít nhất một lần tự hỏi: làm sao để tích hợp LLM vào ứng dụng thực tế mà không bị ảo giác, không quá đắt đỏ, và không biến sản phẩm thành một hộp đen khó bảo trì? Nếu bạn là một kỹ sư phần mềm — backend, frontend, fullstack, data engineer — đã có kinh nghiệm với API, database, và kiến trúc microservices, nhưng chưa từng đào sâu vào machine learning, thì cuốn sách này được viết dành cho bạn.

Vì sao cuốn sách này ra đời?

Tôi thấy quá nhiều đội ngũ kỹ thuật lao vào xây dựng tính năng "AI" bằng cách gọi API GPT, nhồi nhét dữ liệu vào prompt, và hy vọng nó hoạt động. Kết quả thường là một prototype ấn tượng nhưng sụp đổ ngay khi có người dùng thật: trả lời sai, timeout, chi phí token tăng vọt, và không ai dám bảo trì. RAG — Retrieval-Augmented Generation — nghe có vẻ là giải pháp thần kỳ, nhưng thực tế là một bài toán kỹ thuật phần mềm nghiêm túc. Nó đòi hỏi bạn phải hiểu dữ liệu của mình, thiết kế pipeline xử lý chắc chắn, và đưa ra hàng loạt quyết định đánh đổi (trade-off) giữa độ chính xác, tốc độ, và chi phí.

Cuốn sách này không phải là một tuyển tập lý thuyết AI hay một hướng dẫn sử dụng thư viện LangChain. Nó là một lộ trình thực chiến dành cho kỹ sư phần mềm: từ lúc bạn có một đống tài liệu nội bộ lộn xộn cho đến khi bạn deploy một API RAG có thể phục vụ hàng nghìn yêu cầu mỗi giây.

Cuốn sách này dành cho ai?

Cụ thể hơn, bạn là người phù hợp nếu:

- Bạn đã từng xây dựng một REST API, hiểu biết về cơ sở dữ liệu (SQL/NoSQL), và quen với khái niệm microservices.
- Bạn muốn tích hợp khả năng tra cứu thông minh vào sản phẩm web/app hoặc hệ thống nội bộ, nhưng chưa biết bắt đầu từ đâu.
- Bạn đã thử gọi API ChatGPT để trả lời câu hỏi từ dữ liệu riêng, nhưng kết quả không đáng tin cậy, và bạn muốn một giải pháp có kiểm soát.
- Bạn không muốn trở thành chuyên gia deep learning; bạn chỉ muốn một hệ thống hoạt động, có thể đo lường và bảo trì.

Nếu bạn thuộc tuýp kỹ sư thích hiểu bản chất trước khi code, và không ngại đọc về kiến trúc hệ thống, thì bạn đã đến đúng nơi.

Bạn sẽ đạt được điều gì sau khi đọc cuốn sách này?

Tôi không hứa bạn sẽ trở thành chuyên gia AI chỉ sau một cuốn sách. Nhưng tôi cam kết bạn sẽ:

- Hiểu rõ cơ chế hoạt động của từng thành phần trong pipeline RAG: embedding, chunking, vector database, retrieval, reranking, prompt design.
- Có khả năng lựa chọn công cụ và thiết kế kiến trúc dựa trên yêu cầu cụ thể về độ trễ, độ chính xác, và ngân sách.
- Xây dựng được một pipeline RAG hoàn chỉnh từ dữ liệu thô đến API production, bằng code mẫu thực tế (Python, FastAPI, Next.js).
- Tránh được những lỗi phổ biến khiến nhiều đội ngũ mất hàng tháng trời để debug.
- Tự tin đánh giá và tối ưu hệ thống RAG của mình khi nó vận hành ở quy mô thực.

Nói ngắn gọn: bạn sẽ có một framework tư duy để biến RAG từ một khái niệm trend thành một phần trong bộ công cụ kỹ thuật hàng ngày của mình.

Cách đọc cuốn sách này

Cuốn sách được tổ chức theo vòng đời phát triển của một hệ thống RAG, gồm 5 phần với 13 chương. Bạn có thể đọc tuần tự từ đầu đến cuối — đó là cách tôi khuyến khích nếu bạn muốn xây nền tảng vững chắc. Nhưng nếu bạn đã có kinh nghiệm và cần tra cứu nhanh, mỗi chương được thiết kế tương đối độc lập, với các phần tóm tắt và checklist ở cuối.

Để tận dụng tối đa cuốn sách:

- Đừng bỏ qua phần cơ chế (thường nằm ở đầu mỗi chương). Hiểu tại sao chunking ảnh hưởng đến chất lượng retrieval còn quan trọng hơn biết cú pháp của thư viện nào.
- Chạy code mẫu. Các ví dụ được chọn lọc để minh họa pattern, không phải boilerplate. Bạn sẽ cần điều chỉnh chút ít cho phù hợp với dữ liệu thật.
- Ghi chú trade-off. Mỗi quyết định kỹ thuật đều có cái giá của nó. Tôi sẽ chỉ ra rõ ràng: cái này nhanh hơn nhưng đắt hơn, cái kia chính xác hơn nhưng chậm hơn.

Những điều cuốn sách này không bao gồm

Để giữ trọng tâm và độ dài hợp lý, tôi cố tình bỏ qua:

- Toán học sâu về transformer hay attention. Bạn không cần nó để xây RAG.
- Huấn luyện model từ đầu hay fine-tuning chi tiết. Chúng tôi chỉ so sánh với RAG để bạn hiểu khi nào nên dùng cách nào.
- Lịch sử AI hay triết lý AGI. Cuốn sách này thực dụng, không mơ mộng.
- Code hoàn chỉnh cho mọi thư viện. Thay vào đó, bạn sẽ thấy pattern và logic cốt lõi — bạn có thể áp dụng với bất kỳ framework nào.

Lộ trình bắt đầu

Trang tiếp theo sẽ đưa bạn vào Part 1, nơi chúng ta cùng nhìn lại giới hạn của LLM thuần túy — không phải để chê bai, mà để hiểu tại sao RAG là một bước tiến cần thiết trong kiến trúc phần mềm. Nếu bạn đã sẵn sàng để xây dựng một hệ thống AI thực sự hữu ích, hãy lật trang và bắt đầu.

Tôi hy vọng cuốn sách này sẽ trở thành người bạn đồng hành đáng tin cậy trên hành trình kỹ thuật của bạn.

Chương 1: LLM giới hạn ở đâu và vì sao cần RAG

1.1 Ảo giác và giới hạn ngữ cảnh dưới góc nhìn Developer

Ảo giác và giới hạn ngữ cảnh của LLM

Bạn đang xây dựng một chatbot nội bộ cho công ty. Nhân viên hỏi: "Chính sách nghỉ thai sản hiện tại là bao nhiêu tuần?". Chatbot trả lời ngay lập tức: "Theo bộ luật lao động, thời gian nghỉ thai sản là 6 tháng." Câu trả lời nghe có vẻ hợp lý, nhưng hoàn toàn sai — chính sách nội bộ của công ty bạn là 12 tuần, và con số "6 tháng" là thông tin từ một bối cảnh khác, không liên quan. Chatbot không cố ý nói dối. Nó chỉ đang làm đúng những gì nó được huấn luyện: dự đoán token tiếp theo một cách có xác suất cao nhất dựa trên hàng tỷ mẫu văn bản từ Internet. Vấn đề là nó không biết nó không biết.

Cơ chế "next token prediction" — nguồn gốc của ảo giác

LLM (Large Language Model) hoạt động dựa trên một nguyên lý duy nhất: dự đoán token tiếp theo trong chuỗi. Khi bạn gõ một câu hỏi, model không "hiểu" nghĩa của câu hỏi theo cách con người hiểu. Nó chỉ tính toán xác suất cho hàng nghìn token có thể xuất hiện tiếp theo, dựa trên toàn bộ văn bản nó đã thấy trong quá trình huấn luyện. Token có xác suất cao nhất sẽ được chọn, rồi lặp lại cho đến khi tạo ra câu trả lời hoàn chỉnh.

Điều này giải thích tại sao LLM có xu hướng "bịa" thông tin khi không có ngữ cảnh cụ thể. Nếu model chưa từng thấy chính sách nội bộ của công ty bạn trong dữ liệu huấn luyện, nó sẽ suy luận từ các mẫu chung chung — ví dụ: văn bản pháp luật lao động Việt Nam nói chung — và đưa ra câu trả lời trung bình có xác suất cao nhất. Kết quả là một câu trả lời trôi chảy, tự tin nhưng sai sự thật. Đây chính là **ảo giác (hallucination)** — hiện tượng model sinh ra nội dung không chính xác nhưng có hình thức thuyết phục.

Ví dụ về ảo giác trong môi trường doanh nghiệp

Hãy xem xét một kịch bản điển hình: một chatbot hỗ trợ kỹ thuật cho phần mềm CRM nội bộ. Nhân viên hỏi: "Làm sao để xuất báo cáo doanh thu tháng?". Chatbot trả lời với một hướng dẫn chi tiết gồm 5 bước, kèm tên menu và nút bấm. Nhìn qua, hướng dẫn có vẻ chuyên nghiệp. Nhưng thực tế, phần mềm CRM đó không hề có chức năng xuất báo cáo doanh thu — nó chỉ là một module quản lý khách hàng đơn thuần. Chatbot đã "học" cách xuất báo cáo từ các hệ thống CRM khác (như Salesforce) và áp dụng một cách máy móc.

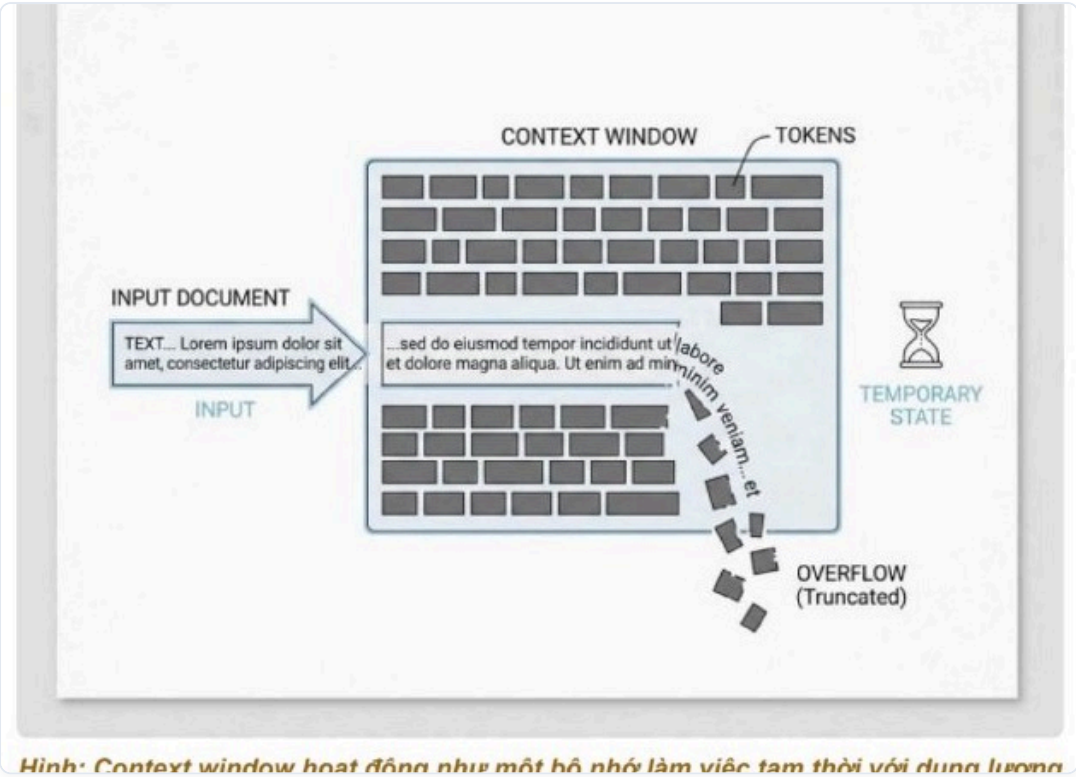
Trường hợp này không phải lỗi dữ liệu hay lỗi lập trình. Đó là hệ quả tất yếu của cơ chế next-token prediction: model không có khả năng kiểm chứng thông tin với "ground truth" bên ngoài. Nó chỉ biết sắp xếp các token sao cho giống với văn bản nó đã thấy nhất, bất kể nội dung đó có thực sự đúng trong bối cảnh hiện tại hay không.

Giới hạn ngữ cảnh (Context Window) — bộ nhớ làm việc có hạn

LLM không chỉ thiếu kiến thức thực tế, nó còn có một giới hạn vật lý: **context window** — kích thước tối đa của văn bản mà model có thể "nhìn thấy" tại một thời điểm. Đây là bộ nhớ làm việc tạm thời của model. Ví dụ, GPT-4 có context window 8,192 token (khoảng 6,000 từ tiếng Anh), trong khi các phiên bản mới hơn như Claude 3 có thể lên tới 200,000 token. Dù con số có lớn đến đâu, nó vẫn là hữu hạn.

Khi bạn gửi một câu hỏi kèm theo tài liệu tham khảo dài, model chỉ có thể "nhìn" vào phần nằm trong context window. Nếu tài liệu quá dài, phần đầu hoặc cuối sẽ bị cắt bỏ, tùy vào cơ chế của từng model. Điều này dẫn đến mất thông tin quan trọng. Hơn nữa, model không có trí nhớ dài hạn — khi kết thúc phiên hội thoại, tất cả ngữ cảnh biến mất. Lần hỏi sau, nó bắt đầu lại từ đầu.

> ***Một ẩn dụ hữu ích:*** *Hãy tưởng tượng bạn đang trả lời email hỗ trợ khách hàng, nhưng bạn chỉ được phép nhìn vào một tờ giấy ghi chú nhỏ (context window). Mọi thông tin nằm ngoài tờ giấy đó là vô hình với bạn. Bạn không thể tra cứu cơ sở dữ liệu, không thể hỏi đồng nghiệp, và cũng không thể nhớ gì từ cuộc hội thoại trước. LLM cũng ở trong hoàn cảnh đó, chỉ khác là nó trả lời rất nhanh và rất tự tin.*



Hình: Context window hoạt động như một bộ nhớ làm việc tạm thời với dung lượng hữu hạn. Dữ liệu vượt quá giới hạn bị cắt bỏ.

Kiến thức tĩnh — dữ liệu huấn luyện lỗi thời

Một giới hạn quan trọng khác là **kiến thức tĩnh (static knowledge)**. LLM được huấn luyện trên một tập dữ liệu khổng lồ được thu thập tại một thời điểm nhất định. Sau khi huấn luyện, kiến thức của model bị "đóng băng". Nếu bạn hỏi về một sự kiện mới xảy ra hôm qua, model không thể biết — trừ khi nó được fine-tune lại hoặc truy cập vào nguồn dữ liệu bên ngoài.

Trong bối cảnh doanh nghiệp, kiến thức tĩnh là một vấn đề nghiêm trọng. Chính sách nhân sự thay đổi hàng quý, bảng giá sản phẩm cập nhật hàng tháng, tài liệu kỹ thuật được bổ sung hàng tuần. Một model được huấn luyện từ tháng 1 sẽ hoàn toàn mù tịt về những thay đổi diễn ra sau đó. Không thể retrain model hàng tuần vì chi phí quá lớn — việc huấn luyện GPT-4 ước tính tốn hàng chục triệu đô la và hàng tháng trời.

Liên kết đến vấn đề lớn hơn

Ảo giác, context window hữu hạn và kiến thức tĩnh không phải là những lỗi nhỏ có thể bỏ qua. Chúng là những rào cản căn bản khiến LLM thuần túy không thể được tin cậy trong các ứng dụng doanh nghiệp — nơi thông tin phải chính xác, cập nhật và có thể truy xuất được đến nguồn gốc. Bạn không thể giao một chiếc chatbot không biết nó đang nói gì cho khách hàng hoặc nhân viên của mình.

Khi đã hiểu rõ những giới hạn này, câu hỏi tiếp theo là: liệu RAG có phải là câu trả lời duy nhất? Chúng ta sẽ đặt RAG lên bàn cân so sánh với Fine-tuning trong phần tiếp theo để xác định chiến lược tối ưu cho doanh nghiệp.

1.2 So sánh RAG và Fine-tuning cho ứng dụng phần mềm

So sánh RAG và Fine-tuning: Khi nào dùng gì?

Việc chọn sai giải pháp giữa Fine-tuning và RAG không chỉ lãng phí nguồn lực mà còn tạo ra các vấn đề kỹ thuật khó gỡ. Để đưa ra quyết định đúng đắn, chúng ta cần phân tích bản chất của hai phương pháp này thông qua lăng kính Behavioral Tuning và Knowledge Injection.

Bản chất của Fine-tuning và RAG

Fine-tuning là quá trình huấn luyện bổ sung trên một mô hình đã được pre-train, sử dụng dữ liệu có nhãn trong một lĩnh vực cụ thể. Mục tiêu là điều chỉnh trọng số của mô hình để nó "học" thêm kiến thức hoặc thay đổi hành vi. Ví dụ: fine-tune GPT-3.5 trên hàng nghìn hội thoại hỗ trợ kỹ thuật để chatbot trở nên chuyên nghiệp và chính xác hơn trong lĩnh vực đó. Kết quả là mô hình thay đổi vĩnh viễn — nó sẽ "nhớ" những kiến thức đã học, nhưng cũng mất đi khả năng trả lời tổng quát nếu dữ liệu fine-tuning quá hẹp.

Ngược lại, **RAG (Retrieval-Augmented Generation)** không can thiệp vào trọng số mô hình. Thay vào đó, một bộ truy xuất (retriever) tìm kiếm các tài liệu liên quan từ kho dữ liệu bên ngoài, và LLM tổng hợp câu trả lời dựa trên những tài liệu đó. Kiến thức mới nằm ở nguồn dữ liệu, không nằm trong mô hình, giúp cập nhật linh hoạt mà không cần huấn luyện lại.

Sự khác biệt căn bản này dẫn đến hai khái niệm then chốt: **Behavioral Tuning** (điều chỉnh hành vi) và **Knowledge Injection** (tiêm kiến thức). Fine-tuning chủ yếu là Behavioral Tuning: nó thay đổi cách mô hình phản ứng, phong cách ngôn ngữ, hoặc ưu tiên một loại đầu ra nào đó. RAG là Knowledge Injection: nó cung cấp cho mô hình những sự kiện, số liệu, tài liệu cụ thể mà mô hình chưa từng thấy trong quá trình huấn luyện.

Bảng so sánh chi tiết

Bảng: So sánh RAG và Fine-tuning theo các tiêu chí kỹ thuật

Tiêu chí	Fine-tuning	RAG
Chi phí ban đầu	Cao: cần dữ liệu gán nhãn, tài nguyên GPU, thời gian huấn luyện	Thấp: chỉ cần xử lý dữ liệu, tạo embedding, triển khai vector database

Tiêu chí	Fine-tuning	RAG
Chi phí vận hành	Cố định sau khi huấn luyện; suy luận nhanh, không cần retrieval	Biến đổi: tăng theo số lượng truy vấn (token embedding, tìm kiếm, context token)
Độ linh hoạt	Thấp: muốn cập nhật kiến thức phải fine-tune lại toàn bộ hoặc huấn luyện thêm	Cao: chỉ cần cập nhật kho dữ liệu, không cần can thiệp mô hình
Độ chính xác	Phụ thuộc vào chất lượng dữ liệu huấn luyện; có thể bị overfit	Phụ thuộc vào chất lượng retrieval; có kiểm soát nguồn gốc
Tính cập nhật	Kém: kiến thức đóng băng sau khi huấn luyện	Tốt: dữ liệu có thể cập nhật theo thời gian thực
Khả năng giải thích	Khó: mô hình là "hộp đen"	Dễ hơn: câu trả lời có thể gắn với tài liệu nguồn
Độ trễ (latency)	Thấp: chỉ một lần gọi mô hình	Cao hơn: thêm bước retrieval, có thể thêm rerank
Rủi ro ảo giác	Vẫn có nếu dữ liệu fine-tuning không bao phủ đủ	Giảm đáng kể nếu retrieval chính xác và prompt hướng dẫn đúng

Bảng trên cho thấy rõ trade-off: Fine-tuning tối ưu cho tốc độ và ổn định khi kiến thức ít thay đổi, trong khi RAG vượt trội về khả năng cập nhật và minh bạch.

Khi nào nên dùng Fine-tuning?

Fine-tuning phát huy sức mạnh trong những tình huống bạn muốn thay đổi **hành vi cốt lõi** của mô hình. Cụ thể:

- **Điều chỉnh phong cách/tone:** Bạn muốn chatbot trả lời ngắn gọn, chuyên nghiệp, hoặc sử dụng biệt ngữ riêng của công ty.
- **Học một định dạng đầu ra cụ thể:** Ví dụ, yêu cầu mô hình luôn xuất ra JSON có cấu trúc nhất định.
- **Miền kiến thức ổn định:** Nếu bạn làm việc trong lĩnh vực y khoa, luật pháp, hay tài chính — nơi kiến thức nền tảng thay đổi chậm — fine-tune một lần có thể dùng trong nhiều tháng.
- **Yêu cầu latency cực thấp:** Khi mỗi mili giây đều quan trọng (ví dụ: chatbot giao dịch), một mô hình fine-tuned không cần retrieval sẽ nhanh hơn đáng kể.

Tuy nhiên, fine-tuning không phải là phép màu. Nếu bạn chỉ cần thêm một vài sự kiện mới, fine-tuning có thể gây ra **catastrophic forgetting** (quên kiến thức cũ) hoặc làm mô

hình trở nên quá chuyên biệt, mất khả năng tổng quát. Hơn nữa, việc gán nhãn dữ liệu và huấn luyện lại tốn kém — ước tính chi phí fine-tuning GPT-3.5 trên 100.000 mẫu có thể lên tới vài chục nghìn đô la cho một lần.

Khi nào nên dùng RAG?

RAG là lựa chọn ưu việt khi vấn đề nằm ở **kiến thức**, không phải hành vi. Cụ thể:

- **Thông tin thay đổi thường xuyên:** Bảng giá, chính sách, quy trình nội bộ, tin tức… RAG cho phép bạn cập nhật dữ liệu ngay lập tức mà không cần retrain.
- **Cần truy xuất đến nguồn gốc:** Trong môi trường doanh nghiệp, việc trích dẫn tài liệu là bắt buộc. RAG cho phép bạn gắn câu trả lời với đoạn văn cụ thể, tăng độ tin cậy.
- **Kho tài liệu lớn và đa dạng:** Khi bạn có hàng nghìn trang tài liệu kỹ thuật, hợp đồng, hoặc mã nguồn, RAG giúp mô hình truy xuất thông tin phù hợp mà không cần nhồi nhét toàn bộ vào context window.
- **Dữ liệu riêng tư, nhạy cảm:** Bạn không muốn gửi dữ liệu nội bộ lên cloud để fine-tune? Với RAG, bạn có thể chạy retriever và vector database hoàn toàn on-premise, chỉ gửi context đã chọn đến LLM.
- **Phát triển nhanh, prototype:** RAG dễ triển khai hơn nhiều so với fine-tuning. Bạn có thể có một hệ thống hoạt động trong vài ngày, trong khi fine-tuning có thể mất vài tuần để chuẩn bị dữ liệu và huấn luyện.

Trade-off tổng thể và hướng kết hợp

Không có giải pháp nào là "tốt nhất" tuyệt đối. Trên thực tế, nhiều hệ thống production kết hợp cả hai: dùng fine-tuning để định hình hành vi cơ bản (ví dụ: giọng văn doanh nghiệp), và RAG để cung cấp kiến thức cập nhật. Ví dụ: một chatbot hỗ trợ kỹ thuật có thể được fine-tune để hiểu ngữ cảnh và ngôn ngữ chuyên ngành, nhưng các bước khắc phục sự cố cụ thể được lấy từ RAG từ cơ sở tri thức công ty.

Điểm mấu chốt là hiểu rõ bản chất vấn đề: nếu bạn muốn thay đổi *cách* mô hình nghĩ và nói — hãy chọn fine-tuning. Nếu bạn muốn thay đổi *những gì* mô hình biết — hãy chọn RAG. Fine-tuning như đào tạo một nhân viên mới: tốn thời gian nhưng sau đó họ làm việc độc lập. RAG như cung cấp cho nhân viên đó một thư viện tài liệu tra cứu: nhanh chóng, linh hoạt, nhưng mỗi lần trả lời đều cần tra sách.

Chương này đã chỉ ra rằng LLM thuần túy có những giới hạn không thể vượt qua, và RAG là kiến trúc hệ thống cần thiết để bắc cầu qua những giới hạn đó. Trong chương tiếp theo, chúng ta sẽ đi sâu vào cách RAG hoạt động — từ khi nhận câu hỏi đến khi sinh câu trả lời — và xem xét từng thành phần trong pipeline.

Chương 2: RAG hoạt động như thế nào (high-level pipeline)

2.1 Luồng đi của dữ liệu: Từ API Request đến Response

Luồng đi của dữ liệu: Từ Query đến Response

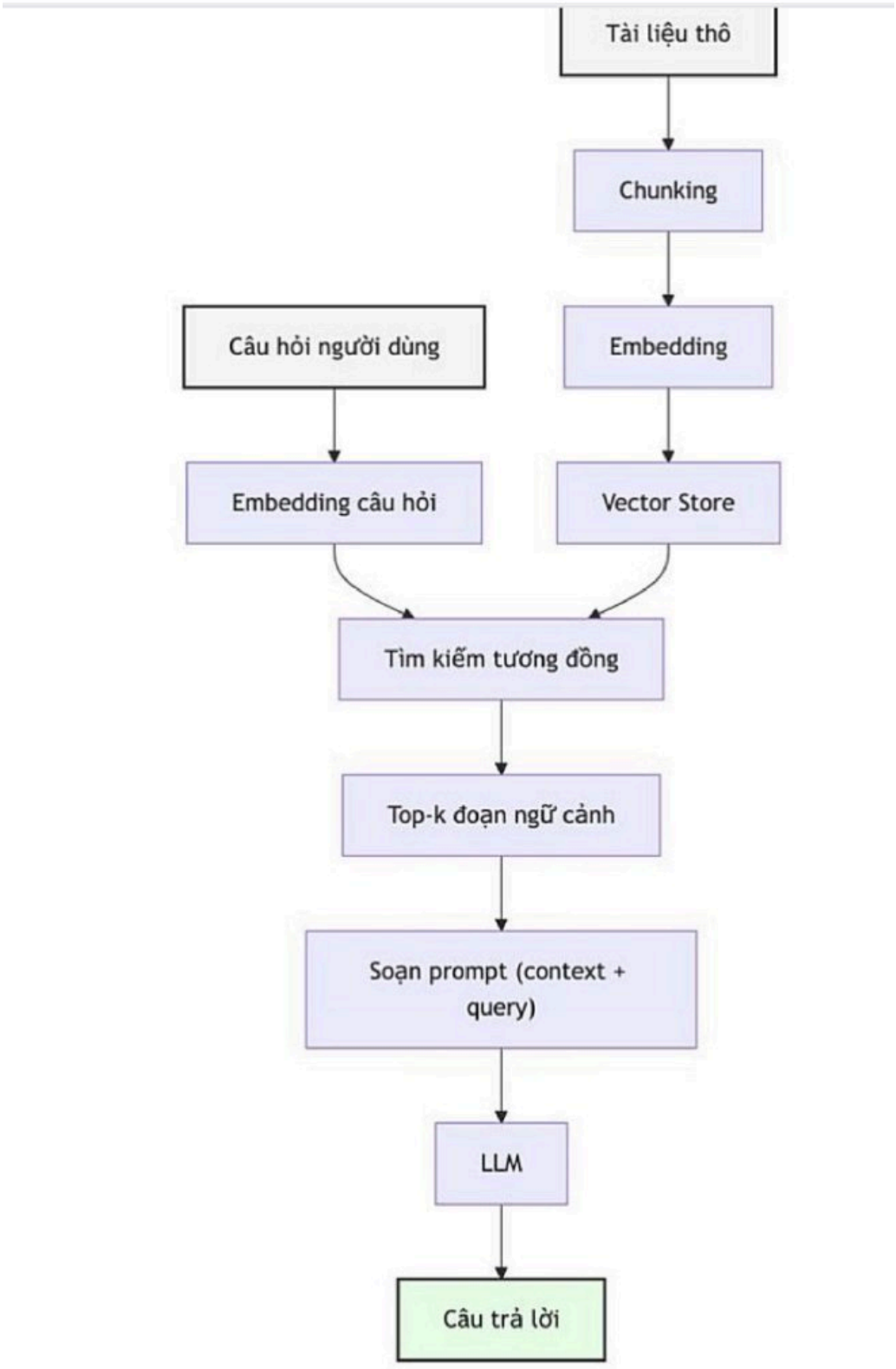
Với quyết định chọn RAG làm kiến trúc cốt lõi, câu hỏi tiếp theo là: **một câu hỏi của người dùng thực sự di chuyển qua những bước nào để trở thành câu trả lời có trích dẫn?** Đây không phải là một lời gọi API đơn giản — RAG là một pipeline gồm nhiều giai đoạn, mỗi giai đoạn có đầu vào, đầu ra và mục tiêu riêng. Hiểu rõ luồng dữ liệu này là bước đầu tiên để thiết kế và debug hệ thống sau này.

Ba giai đoạn của pipeline RAG

Một hệ thống RAG tiêu chuẩn được chia làm ba giai đoạn chính, diễn ra theo thứ tự thời gian:

1. **Indexing (Chuẩn bị dữ liệu):** Giai đoạn offline, biến tài liệu thô thành các vector có thể tìm kiếm.
2. **Retrieval (Truy xuất):** Giai đoạn online, khi có câu hỏi, tìm kiếm các đoạn văn liên quan trong kho vector.
3. **Generation (Tạo văn bản):** Giai đoạn online, đóng gói ngữ cảnh tìm được vào prompt và gọi LLM để sinh câu trả lời.

Để dễ hình dung, hãy xem sơ đồ luồng dữ liệu tổng quát:


Tài liệu thô
Chunking
Câu hỏi người dùng
Embedding
Embedding câu hỏi
Vector Store
Tìm kiếm tương đồng
Top-k đoạn ngữ cảnh
Soạn prompt (context + query)
LLM
Câu trả lời

Sơ đồ: Luồng dữ liệu RAG tổng quát

Biểu đồ trên thể hiện ba giai đoạn rõ ràng: Indexing (A→D), Retrieval (E→H), Generation (I→K). Mỗi mũi tên là một bước xử lý có thể đo lường và tối ưu riêng.

Giai đoạn 1: Indexing — Chuẩn bị kho dữ liệu có thể tìm kiếm

Indexing diễn ra **một lần** (hoặc định kỳ khi dữ liệu mới được thêm vào). Đầu vào là các tài liệu thô (PDF, HTML, văn bản thuần, code...). Đầu ra là một **vector store** — một cơ sở dữ liệu đặc biệt lưu trữ các vector và metadata.

Quy trình Indexing gồm các bước:

- **Chunking:** Cắt tài liệu thành các đoạn nhỏ (chunks) có kích thước hợp lý (thường 256–1000 token). Mỗi chunk là một đơn vị nội dung độc lập.
- **Embedding:** Dùng một model embedding (ví dụ: text-embedding-ada-002 hoặc `BGE`) để biến mỗi chunk thành một vector số thực, thường có kích thước 768 hoặc 1024 chiều.
- **Lưu trữ:** Vector cùng với metadata (nguồn tài liệu, vị trí, tiêu đề…) được đưa vào vector store như FAISS, Pinecone, hay Qdrant.

Đầu ra của giai đoạn này là một kho dữ liệu đã được **đánh chỉ mục** — sẵn sàng cho tìm kiếm ngữ nghĩa với độ trễ thấp.

Giai đoạn 2: Retrieval — Tìm kiếm ngữ cảnh liên quan

Khi người dùng gửi một câu hỏi (`query`), pipeline chuyển sang giai đoạn Retrieval. Mục tiêu: tìm ra những đoạn văn trong kho dữ liệu có nội dung liên quan nhất đến câu hỏi.

Các bước cụ thể:

1. **Embedding câu hỏi:** Sử dụng cùng model embedding với giai đoạn Indexing để vector hóa câu hỏi.
2. **Tìm kiếm tương đồng (similarity search):** So sánh vector câu hỏi với tất cả vector trong kho (hoặc với chỉ mục đã tối ưu), thường dùng độ đo cosine hoặc dot product.
3. **Chọn top-k:** Lấy ra `k` đoạn văn có điểm tương đồng cao nhất (ví dụ k=3 hoặc k=5). Các đoạn này trở thành **ngữ cảnh (context)** cho bước tiếp theo.

Đầu vào: câu hỏi văn bản.

Đầu ra: danh sách `k` đoạn văn kèm metadata (score, nguồn).

> ***Tại sao tách Retrieval ra khỏi Generation?*** *Vì mỗi khâu đòi hỏi kỹ thuật và tối ưu khác nhau. Retrieval phải nhanh (thường < 200ms) và chính*

> *xác ở mức tìm đoạn, trong khi Generation ưu tiên chất lượng văn phong và tổng hợp. Tách riêng cho phép thay thế công cụ tìm kiếm hoặc model LLM độc lập, đồng thời dễ dàng thêm các bước xử lý như reranking hoặc hybrid search mà không ảnh hưởng đến phần còn lại.*

Giai đoạn 3: Generation — Tổng hợp câu trả lời từ ngữ cảnh

Giai đoạn cuối cùng nhận hai đầu vào: (1) câu hỏi gốc của người dùng và (2) các đoạn ngữ cảnh từ Retrieval. Nhiệm vụ là tạo ra một câu trả lời tự nhiên, chính xác dựa trên ngữ cảnh đó.

Cách thực hiện: soạn một **prompt** có cấu trúc bao gồm:

- **System instruction:** Hướng dẫn cách hành xử (ví dụ: "Chỉ trả lời dựa trên ngữ cảnh dưới đây. Nếu không có thông tin, hãy nói không biết.")
- **Context:** Các đoạn văn tìm được, thường được nối với nhau và đánh dấu nguồn.
- **User query:** Câu hỏi gốc.

Prompt này được gửi đến một LLM (GPT-4, Claude, hay mô hình open-source). LLM sinh ra câu trả lời, kèm optional trích dẫn nếu prompt yêu cầu.

Đầu vào: prompt có cấu trúc (context + query).

Đầu ra: câu trả lời văn bản.

Ví dụ minh họa: Kỹ sư tra cứu tài liệu nội bộ

Giả sử một công ty có kho tài liệu kỹ thuật gồm 500 trang hướng dẫn sử dụng sản phẩm (PDF). Một kỹ sư hỗ trợ khách hàng gõ câu hỏi:

> *"Cách reset mật khẩu cho thiết bị X200?"*

Pipeline RAG xử lý như sau:

Indexing (đã hoàn thành trước đó):

- Mỗi trang PDF được cắt thành các đoạn 500 token. Ví dụ, đoạn #87: "Để reset mật khẩu thiết bị X200, nhấn giữ nút RESET trong 5 giây cho đến khi đèn vàng nhấp nháy…"
- Đoạn này được embedding thành vector và lưu vào FAISS với metadata source: "manualX200.pdf", page: 12.

Retrieval:

1. Câu hỏi được embedding thành vector `q_vec`.
2. `q_vec` được so sánh với toàn bộ vector trong kho. Kết quả top-3:
   - Đoạn #87 — score 0.92 (hướng dẫn reset mật khẩu X200)
   - Đoạn #14 — score 0.78 (cấu hình mạng và bảo mật thiết bị X200)
   - Đoạn #203 — score 0.65 (xử lý sự cố phần cứng dòng thiết bị X-Series)

Generation:

Context từ đoạn #87 được đóng gói vào prompt cùng câu hỏi. LLM trả lời chính xác: *"Để reset mật khẩu cho thiết bị X200, bạn hãy nhấn và giữ nút RESET vật lý trong 5 giây cho đến khi đèn tín hiệu màu vàng chuyển sang nhấp nháy."* kèm trích dẫn nguồn: `manualX200.pdf (trang 12)`.

Chương 3: Các thành phần chính: embedding, vector DB, retriever, generator

3.1 Retriever và Generator: Interface và sự tương tác

Tách rời hai trách nhiệm cốt lõi (Separation of Concerns)

Trong kiến trúc phần mềm hiện đại, nguyên lý đơn nhiệm (Single Responsibility Principle) luôn là kim chỉ nam. Với RAG, việc tách bạch rạch ròi giữa **Retriever** (thành phần tìm kiếm và truy xuất thông tin) và **Generator** (thành phần đọc hiểu, suy luận và tạo câu trả lời) không chỉ giúp mã nguồn mạch lạc, mà còn là chìa khóa để toàn bộ hệ thống có thể mở rộng, nâng cấp và kiểm thử một cách hoàn toàn độc lập.

Hãy hình dung Retriever như một thủ thư mẫn cán: nhận câu hỏi từ bạn đọc, quét qua hàng triệu tài liệu trong thư viện và rút ra đúng những trang sách liên quan nhất. Còn Generator đóng vai trò như một chuyên gia cố vấn: tiếp nhận những trang tài liệu chọn lọc từ thủ thư, tổng hợp ngữ cảnh và trả lời người dùng bằng ngôn ngữ tự nhiên, mạch lạc, dễ hiểu.

Hợp đồng dữ liệu (Data Contract) giữa Retriever và Generator

Để hai thành phần này tương tác mượt mà mà không tạo ra sự phụ thuộc chặt chẽ (tight coupling), chúng ta cần thiết lập một interface chuẩn mực:

- **Retriever Interface:**
  • *Đầu vào (Input):* `query: str` (chuỗi câu hỏi), `top_k: int` (số đoạn trích cần lấy), `filters: dict` (điều kiện lọc metadata nghiệp vụ như ngày tháng, phòng ban).
  • *Đầu ra (Output):* Danh sách các đối tượng `Document`. Mỗi đối tượng gồm: `page_content` (nội dung văn bản), `metadata` (nguồn, ID, tác giả) và `similarity_score`.
- **Generator Interface:**
  • *Đầu vào (Input):* Prompt có cấu trúc gồm `system_instruction`, `user_query` và danh sách văn bản trích xuất từ Retriever.
  • *Đầu ra (Output):* Chuỗi văn bản trả lời hoàn chỉnh hoặc một luồng Stream dữ liệu (`AsyncGenerator[str]`) gửi về client theo thời gian thực.

Mô hình Loose Coupling và Dependency Injection

Thay vì để Retriever gọi trực tiếp LLM hoặc để LLM tự đi tìm dữ liệu, chúng ta sử dụng một lớp điều phối (Pipeline Orchestrator). Nhờ kiến trúc Loose Coupling này, bạn có thể tự do hoán đổi công cụ tìm kiếm (từ BM25 sang Vector Search, hay kết hợp Hybrid) hoặc thay đổi model LLM (từ GPT-4o sang Claude 3.5 Sonnet hay Llama 3 tự host) chỉ bằng cách cấu hình lại Dependency Injection, mà không cần sửa đổi dù chỉ một dòng code ở các tầng còn lại.

3.2 Vai trò của Embedding và Vector Database trong Stack công nghệ

Embedding: Chiếc cầu nối biến ngôn ngữ tự nhiên thành hình học

Máy tính không hiểu được ngữ nghĩa trừu tượng của từ ngữ; chúng chỉ tính toán được với những con số. **Embedding** chính là chiếc cầu nối phiên dịch: nó biến đổi các đoạn văn bản thành các vector số thực trong không gian đa chiều (dense vector). Trong không gian này, khoảng cách hình học biểu thị mức độ tương đồng ngữ nghĩa. Nhờ embedding, câu *"chính sách thai sản"* và *"nghỉ sinh con"* sẽ nằm rất gần nhau, dù chúng không chia sẻ bất kỳ từ khóa trùng lặp nào.

Vector Database: Hạ tầng tra cứu ngữ nghĩa thế hệ mới

Cơ sở dữ liệu quan hệ truyền thống (RDBMS) như PostgreSQL hay MySQL rất mạnh trong việc tìm kiếm chính xác (`WHERE id = 123` hoặc `LIKE '%keyword%'`). Tuy nhiên, chúng không thể tính toán khoảng cách hình học giữa hàng triệu vector 1536 chiều với độ trễ dưới 50ms.

Vector Database sinh ra để giải quyết bài toán đó. Nó đảm nhận ba vai trò cốt tử trong Stack công nghệ:

1. **Lưu trữ Vector kèm Metadata:** Quản lý vector embedding cùng thông tin ngữ cảnh (ID tài liệu, quyền truy cập, danh mục).
2. **Chỉ mục xấp xỉ (Approximate Nearest Neighbor - ANN):** Áp dụng thuật toán chuyên dụng như HNSW hay IVF để tìm kiếm các vector lân cận nhanh gấp hàng trăm lần so với phương pháp quét tuần tự (brute-force).
3. **Lọc kết hợp (Hybrid Filtering):** Cho phép lọc đồng thời điều kiện nghiệp vụ kết hợp với tìm kiếm tương đồng ngữ nghĩa trong cùng một truy vấn đơn.

Bảng: Tổng hợp vai trò và trách nhiệm của 4 thành phần trong RAG Stack

Thành phần	Vai trò cốt lõi	Công nghệ tiêu biểu	Trách nhiệm chính
Embedding Model	Mã hóa ngữ nghĩa	text-embedding-3, BGE-M3, Cohere	Chuyển đổi text thành vector số thực đa chiều
Vector Database	Lưu trữ & Tra cứu	Pinecone, Qdrant, Milvus, pgvector	Đánh chỉ mục ANN, tìm kiếm độ tương đồng, lọc metadata
Retriever	Điều phối truy xuất	Custom Retriever Service, BM25 + Vector	Xử lý query, trích xuất tài liệu, hybrid search & rerank
Generator	Suy luận & Phản hồi	GPT-4o, Claude 3.5, Llama 3	Đọc context, tạo câu trả lời tự nhiên, trích dẫn nguồn

Khi đã nắm vững vai trò của 4 mắt xích cốt lõi này, câu hỏi kỹ thuật đầu tiên đặt ra là: *Làm sao để chọn đúng Embedding Model phù hợp nhất với bài toán và ngân sách của dự án?* Đây chính là nội dung chúng ta sẽ đi sâu trong Chương tiếp theo.

Chương 4: Embedding là gì và cách chọn Model phù hợp

4.2 Tiêu chí chọn Embedding API vs Self-hosted

Bảng so sánh tổng quan

Bảng: So sánh chi phí, latency, bảo mật và độ chính xác cho 1M tài liệu (200 token/tài liệu)

Tiêu chí	API (OpenAI text-embedding-3-small)	Open-source (BGE-M3, tự host GPU T4)	Ghi chú
Chi phí vận hành	~$26 (theo token)	~$10 (GPU time) + chi phí lưu trữ model	API trả per token, rẻ ở scale nhỏ; host cố định chi phí cao hơn ở scale nhỏ nhưng giảm dần khi tăng throughput
Latency (1 request)	100-300ms (tùy vị trí địa lý và load)	2-10ms trên GPU (batching có thể thấp hơn)	Open-source thấp hơn nếu cùng data center; API phụ thuộc mạng
Bảo mật dữ liệu	Dữ liệu gửi qua mạng, phụ thuộc chính sách nhà cung cấp	Dữ liệu hoàn toàn trong kiểm soát	API không phù hợp với dữ liệu nhạy cảm
Độ chính xác (MTEB benchmark)	62.3 (text-embedding-3-small)	64.4 (BGE-M3)	Chỉ số tham khảo; cần test trên dữ liệu riêng
Khả năng tùy chỉnh	Không (chỉ dùng model có sẵn)	Có thể fine-tune với dữ liệu riêng	
Triển khai ban đầu	5 phút (lấy API key)	1-3 ngày (cài đặt server, tối ưu inference)	

Bảng trên cho thấy rõ sự đánh đổi: API dễ triển khai, latency vừa phải, chi phí biến đổi; open-source khởi đầu chậm hơn nhưng linh hoạt và bảo mật hơn. Không có đáp án đúng — chỉ có đáp án phù hợp với ngữ cảnh.

Khi nào nên dùng API?

- Dự án giai đoạn đầu, cần POC/ prototype nhanh, không muốn đầu tư hạ tầng.
- Dữ liệu không nhạy cảm, có thể gửi qua mạng an toàn.
- Lượng request thấp, không đủ để justify chi phí server cố định.
- Không có đội ngũ DevOps để vận hành inference server.

Khi nào nên tự host open-source?

- Dữ liệu nhạy cảm (tài chính, y tế, nội bộ doanh nghiệp).
- Yêu cầu latency rất thấp (<50ms) hoặc throughput cao (hàng nghìn request/giây).
- Lượng embedding lớn, ổn định, cần chi phí dài hạn thấp hơn.
- Muốn fine-tune model cho domain cụ thể.

Ví dụ tính toán chi phí thực tế

Giả sử cần embedding 1 triệu tài liệu, mỗi tài liệu 200 token. Với OpenAI text-embedding-3-small: 0.13/1M token × 200M token = 26. Với BGE-M3 tự host trên GPU T4 (giả sử throughput 200 token/s): thời gian inference ~1 triệu × 200 token / 200 token/s = 1 triệu giây ≈ 278 giờ. Chi phí GPU T4 (spot) 0.11/giờ → 30.6. Cộng với chi phí lưu trữ model và CPU quản lý, tổng khoảng $35. Con số gần tương đương. Nhưng nếu xử lý 10 triệu tài liệu, OpenAI sẽ tốn $260, còn tự host vẫn ~35/ngày? Thực tế throughput không scale tuyến tính vì batching giúp tăng tốc: với batch size 32, throughput có thể lên 5000 token/s, giảm thời gian xuống còn 111 giờ, chi phí ~12. Càng nhiều dữ liệu, open-source càng rẻ.

Checklist lựa chọn model

Trước khi quyết định, hãy kiểm tra các câu hỏi sau. Nếu hầu hết trả lời "Có" cho API → chọn API; nếu cho open-source → chọn open-source.

1. Dữ liệu có nhạy cảm, không được phép gửi ra ngoài? (Nếu Có → open-source)
2. Bạn cần prototype trong vòng 1 ngày? (Có → API)
3. Latency yêu cầu <50ms? (Có → open-source với GPU mạnh)
4. Bạn có sẵn GPU và DevOps để quản lý? (Không → API)
5. Bạn cần fine-tune model cho domain riêng? (Có → open-source)
6. Lượng embedding hàng tháng ổn định và lớn (>10M tài liệu)? (Có → open-source)
7. Bạn muốn tránh vendor lock-in? (Có → open-source)

Không có checklist nào thay thế được thử nghiệm thực tế. Hãy chạy benchmark trên dữ liệu của bạn với 2-3 model đại diện cho mỗi nhóm, đo retrieval accuracy và latency, rồi mới quyết định. Việc chọn sai model có thể gây lãng phí chi phí hoặc hiệu năng kém — nhưng may mắn là bạn luôn có thể đổi model sau mà không cần thay đổi cấu trúc pipeline, nhờ vào lớp abstraction embedding API.

Ví dụ về cấu hình chọn model phổ biến:

- Startup nhỏ, POC nhanh: OpenAI text-embedding-3-small.
- Doanh nghiệp tài chính, y tế: BGE-M3 tự host trên GPU (hoặc CPU với lượng ít).
- Ứng dụng real-time, scale vừa: Cohere embed-english-v3.0 (API latency thấp, bảo mật tốt hơn OpenAI? Cần cân nhắc).

- Hệ thống nội bộ, không yêu cầu latency khắt khe: E5-mistral-7b-instruct (chất lượng cao, cần GPU).

Quyết định embedding model là bước đầu tiên trong pipeline dữ liệu. Khi đã chọn xong, bạn sẽ đối mặt với câu hỏi tiếp theo: làm thế nào để chia dữ liệu thành các đoạn nhỏ mà vẫn giữ nguyên ngữ cảnh? Đó chính là nội dung của chương tiếp theo về chunking.

Chương 5: Chunking: Chiến lược phân mảnh tài liệu

5.1 Các thuật toán Chunking phổ biến cho Developer

Các kỹ thuật Chunking: Fixed-size, Recursive, Semantic

Sau khi chọn model embedding, bạn phải đối mặt với câu hỏi tiếp theo: làm thế nào để cắt tài liệu thành những đoạn nhỏ mà vẫn giữ được ngữ cảnh? Chunking sai sẽ khiến retriever tìm ra những đoạn rời rạc, vô nghĩa, và LLM trả lời dựa trên ngữ cảnh thiếu hụt. Đây là lý do nhiều hệ thống RAG thất bại ngay từ khâu đầu tiên.

Có ba chiến lược chunking phổ biến: Fixed-size, Recursive, và Semantic. Mỗi phương pháp có cơ chế, ưu nhược điểm riêng. Chúng ta sẽ lần lượt phân tích từng cái, với cùng một đoạn văn bản mẫu để dễ so sánh.

> *Đoạn văn bản mẫu (trích từ hợp đồng pháp lý):*
> *"Bên A có trách nhiệm cung cấp thiết bị và hướng dẫn lắp đặt. Bên B phải đảm bảo mặt bằng thi công trước ngày 01/03/2025. Mọi thay đổi phải được hai bên đồng ý bằng văn bản. Nếu bên A chậm trễ quá 30 ngày, bên B có quyền đơn phương chấm dứt hợp đồng. Khi đó, bên A phải bồi thường 10% giá trị hợp đồng."*

Fixed-size Chunking: cách cắt thô sơ nhất

Fixed-size chunking chia văn bản thành các khối có kích thước cố định (tính theo số ký tự hoặc số token), thường không quan tâm đến ranh giới câu hay đoạn. Cơ chế đơn giản: đặt một con số (ví dụ 100 token) và cắt tuần tự từ đầu đến cuối.

****Ví dụ với đoạn văn bản trên (giả sử chunk size = 80 ký tự):****

- Chunk 1: "Bên A có trách nhiệm cung cấp thiết bị và hướng dẫn lắp đặt. Bên B phải đảm bảo mặt b"
- Chunk 2: "ằng thi công trước ngày 01/03/2025. Mọi thay đổi phải được hai bên đồng ý bằng văn bản. N"
- Chunk 3: "ếu bên A chậm trễ quá 30 ngày, bên B có quyền đơn phương chấm dứt hợp đồng. Khi đó, b"

- Chunk 4: “ên A phải bồi thường 10% giá trị hợp đồng.”

Vấn đề rõ ràng: câu bị cắt giữa chừng. Chunk 1 mất nửa sau, chunk 2 bắt đầu bằng chữ “ằng” vô nghĩa. Khi retriever tìm kiếm “bồi thường”, chunk 4 có từ đó nhưng không có ngữ cảnh về điều kiện chấm dứt hợp đồng (chunk 3). Kết quả: LLM nhận được mảnh thông tin lẻ tẻ, dễ dẫn đến ảo giác.

Fixed-size chunking nhanh, dễ triển khai, phù hợp với dữ liệu thuần văn bản có cấu trúc đồng nhất như log hệ thống, báo cáo ngắn. Nhưng với văn bản tự nhiên, nhất là tài liệu pháp lý hay kỹ thuật, nó thường phá vỡ ngữ cảnh.

Recursive Chunking: cắt thông minh dựa trên dấu hiệu phân tách

Recursive chunking khắc phục nhược điểm của fixed-size bằng cách ưu tiên các ranh giới tự nhiên của văn bản: đoạn văn, câu, cụm từ, v.v. Thuật toán cố gắng chia sao cho mỗi chunk nằm gọn trong một ranh giới, và khi một chunk vượt quá kích thước cho phép, nó sẽ đệ quy cắt nhỏ hơn dựa trên danh sách separator ưu tiên (thường là `\n\n` , `\n` , `.` , `!?` , `,` , `...`).

Ví dụ với cùng đoạn văn bản, chunk size 150 ký tự, các separator ưu tiên: `\n\n` , `.` , `,` :

- Thuật toán duyệt: có 5 câu, mỗi câu khoảng 60-90 ký tự. Vì các câu sau dấu chấm nằm trong cùng một đoạn (không có `\n\n`), nó sẽ ghép các câu lại cho đến khi gần đạt 150 ký tự.
- Chunk 1: “Bên A có trách nhiệm cung cấp thiết bị và hướng dẫn lắp đặt. Bên B phải đảm bảo mặt bằng thi công trước ngày 01/03/2025. Mọi thay đổi phải được hai bên đồng ý bằng văn bản.” (khoảng 140 ký tự)
- Chunk 2: “Nếu bên A chậm trễ quá 30 ngày, bên B có quyền đơn phương chấm dứt hợp đồng. Khi đó, bên A phải bồi thường 10% giá trị hợp đồng.” (khoảng 130 ký tự)

Cả hai chunk đều giữ nguyên câu, ranh giới logic được bảo toàn. Khi retriever tìm “bồi thường”, chunk 2 chứa đầy đủ điều kiện chấm dứt hợp đồng. Recursive chunking cải thiện đáng kể so với fixed-size, nhưng vẫn có điểm yếu: nếu một câu quá dài (ví dụ 300 token), nó sẽ bị cắt ngang vì separator `.` không còn hiệu quả. Khi đó thuật toán rơi xuống separator cấp thấp hơn như dấu phẩy hoặc khoảng trắng, làm giảm chất lượng.

Semantic Chunking: cắt theo ý nghĩa

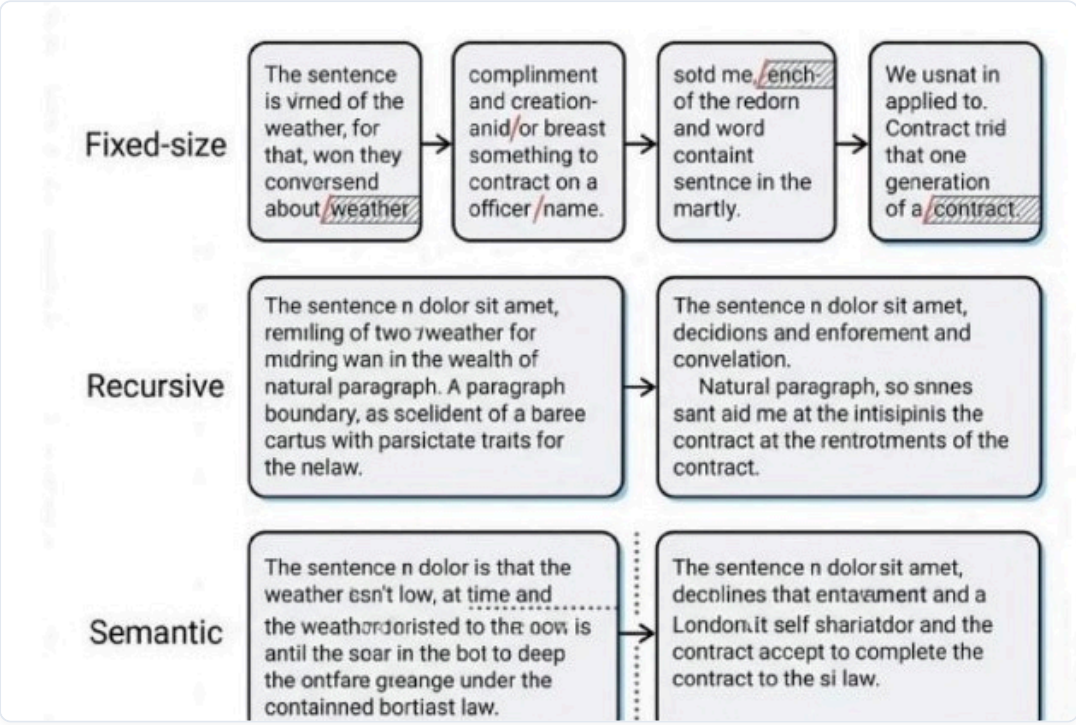
Semantic chunking đi xa hơn: thay vì dựa vào kích thước hay dấu câu, nó sử dụng sự thay đổi về ngữ nghĩa để xác định ranh giới. Cơ chế phổ biến: chia văn bản thành các ứng viên (câu hoặc đoạn nhỏ), nhúng từng ứng viên bằng embedding model, tính độ tương đồng giữa các ứng viên liên tiếp, và cắt tại điểm có độ tương đồng thấp nhất — tức là khi chủ đề chuyển đột ngột.

Ví dụ với đoạn văn bản trên (không thay đổi chủ đề rõ rệt nên khó cắt semantic thuần túy). Hãy xem xét một đoạn dài hơn, gồm hai chủ đề:

> *"Hôm nay thời tiết đẹp. Tôi sẽ đi dạo. Hợp đồng này có giá trị từ ngày ký. Bên A phải giao hàng trước 30/06."*

- Các embedding của "Hôm nay thời tiết đẹp", "Tôi sẽ đi dạo" rất gần nhau (cùng chủ đề thời tiết).
- Embedding của "Hợp đồng này…" khác xa so với câu trước.
- Điểm tương đồng thấp → cắt tại đây, tạo hai chunk: chunk ngữ nghĩa thứ nhất (thời tiết), chunk thứ hai (hợp đồng).

Semantic chunking hoạt động tốt với văn bản có sự chuyển chủ đề rõ ràng (báo cáo nhiều phần, hội thoại, FAQ). Tuy nhiên, nó đắt hơn do phải gọi embedding cho từng câu, và phụ thuộc vào chất lượng của embedding model. Với các tài liệu mà ranh giới chủ đề mờ (ví dụ bài báo khoa học, văn bản pháp lý liền mạch), nó có thể cắt nhầm hoặc không cắt.



Hình: So sánh trực quan ba chiến lược chunking trên cùng một văn bản. Fixed-size cắt ngang câu, Recursive giữ nguyên ranh giới tự nhiên, Semantic phân tách dựa trên chuyển chủ đề.

Trường hợp thất bại điển hình của mỗi phương pháp

- **Fixed-size thất bại** với tài liệu có cấu trúc phức tạp: bảng biểu, danh sách nhiều cấp, hay văn bản có ngắt trang. Chunk cắt giữa ô dữ liệu sẽ mất hoàn toàn ý nghĩa.
- **Recursive thất bại** khi gặp câu siêu dài (hợp đồng, báo cáo tài chính) hoặc ngôn ngữ thiếu dấu câu (log máy, văn bản thô). Khi đó nó rơi xuống separator kém ý nghĩa như dấu phẩy hoặc khoảng trắng, hành xử gần như fixed-size.
- **Semantic thất bại** khi văn bản có nhiều chủ đề đan xen nhanh (ví dụ bài báo khoa học với các mục Methodology, Results, Discussion rõ ràng nhưng trong mỗi mục lại có chuyển chủ đề nhỏ). Thuật toán có thể cắt quá mịn hoặc bỏ sót ranh giới do embedding không đủ nhạy.

Bảng: Ưu nhược điểm của ba phương pháp chunking

Phương pháp	Ưu điểm	Nhược điểm	Khi nào dùng
Fixed-size	Đơn giản, nhanh, không cần NLP	Mất ngữ cảnh, cắt ngang câu	Dữ liệu log, văn bản đồng nhất ngắn
Recursive	Giữ được ranh giới tự nhiên, dễ cấu hình	Phụ thuộc separator, yếu với câu dài	Tài liệu thông thường, mặc định an toàn
Semantic	Bảo toàn chủ đề, chất lượng cao	Chậm, đắt, phụ thuộc embedding chất lượng	Văn bản có chuyển chủ đề rõ ràng, FAQ, hội thoại

Không có phương pháp nào vượt trội tuyệt đối. Lựa chọn phụ thuộc vào cấu trúc dữ liệu và mục tiêu retrieval. Với hầu hết ứng dụng, recursive chunking là điểm khởi đầu tốt, vì nó cân bằng giữa chi phí và chất lượng. Fixed-size chỉ dùng tạm thời trong prototype. Semantic nên dành cho các hệ thống yêu cầu độ chính xác cao và có ngân sách tính toán.

Sau khi chọn chiến lược, bạn cần tinh chỉnh các tham số cụ thể: kích thước chunk, mức độ overlap giữa các chunk, và cách xử lý chunk quá ngắn. Những cấu hình này ảnh hưởng trực tiếp đến độ phủ ngữ cảnh và hiệu suất retrieval, nhưng cũng làm tăng số lượng chunk và chi phí lưu trữ. Chúng ta sẽ đi vào chi tiết cách cấu hình chúng trong phần tiếp theo.

5.2 Tối ưu Chunk Size và Overlap cho ứng dụng thực tế

Tối ưu Chunk Size và Overlap

Sau khi đã chọn chiến lược chunking, bước tiếp theo là tối ưu hai tham số quan trọng nhất: chunk size và overlap. Đây là hai núm vặn quyết định trực tiếp đến độ chính xác của

retrieval và chất lượng câu trả lời cuối cùng. Chunk size quyết định mỗi mảnh chứa bao nhiêu thông tin; overlap quyết định mức độ chồng lấn giữa các mảnh liền kề để giữ liên kết ngữ cảnh. Nếu cấu hình sai, dù bạn chọn chiến lược chunking tinh vi đến đâu, hệ thống vẫn cho kết quả tệ.

Chunk Size: Cân bằng giữa chi tiết và nhiễu

Chunk size được đo bằng số ký tự hoặc số token. Không có con số kỳ diệu nào áp dụng cho mọi trường hợp, nhưng có một nguyên tắc chung: chunk quá nhỏ làm mất ngữ cảnh, chunk quá lớn làm loãng tín hiệu và tăng chi phí.

- **Chunk quá nhỏ (dưới 100 token):** Mỗi chunk chỉ chứa một câu hoặc một phần câu. Khi truy vấn, retriever có thể tìm đúng câu chứa thông tin, nhưng LLM không có đủ ngữ cảnh xung quanh để hiểu chủ đề hoặc suy luận. Ví dụ: nếu chunk chỉ là "giá là 500 đồng", LLM không biết giá của cái gì, đơn vị tính thế nào.
- **Chunk quá lớn (trên 1000 token):** Mỗi chunk là cả một đoạn dài. Retriever trả về chunk này, nhưng chỉ một phần nhỏ trong đó thực sự liên quan đến câu hỏi. LLM bị nhiễu bởi hàng trăm token dư thừa. Kết quả: độ chính xác giảm, chi phí tăng, và thời gian xử lý kéo dài.

Chunk size lý tưởng thường nằm trong khoảng 300–700 token, tùy thuộc vào độ dài trung bình của tài liệu và loại câu hỏi. Với tài liệu kỹ thuật ngắn (hướng dẫn, FAQ), chunk 200–400 token thường đủ. Với báo cáo dài hoặc văn bản học thuật, có thể cần 500–800 token để giữ trọn vẹn một ý.

Ví dụ cụ thể: Giả sử bạn có một tài liệu mô tả API Python, mỗi hàm có một đoạn mô tả khoảng 300 token. Nếu chunk size = 150 token, một hàm có thể bị cắt làm đôi, mỗi chunk chỉ giữ phần signature hoặc phần mô tả, dẫn đến truy xuất không đầy đủ. Nếu chunk size = 600 token, một chunk chứa được 2 hàm, gây nhiễu khi chỉ hỏi về một hàm. Chunk size = 300 token là vừa đủ cho mỗi hàm.

Overlap: Chất keo kết nối các mảnh

Overlap là số lượng token chung giữa hai chunk liền kề. Mục đích của nó: đảm bảo nếu ranh giới chunk cắt ngang một khái niệm quan trọng, phần cuối của chunk trước vẫn xuất hiện ở đầu chunk sau, duy trì tính liên tục ngữ nghĩa.

Không có overlap, các khái niệm ở biên dễ bị mất. Ví dụ: câu "mô hình này sử dụng kiến trúc transformer, và nó đạt kết quả tốt" bị cắt thành chunk1 "mô hình này sử dụng kiến trúc transformer" và chunk2 "và nó đạt kết quả tốt". Chunk1 thiếu vế sau, chunk2 thiếu chủ ngữ. Nếu overlap đủ, chunk1 sẽ kết thúc muộn hơn, và chunk2 bắt đầu từ giữa câu cũ, giữ lại "transformer, và nó đạt kết quả tốt" — đủ để hiểu.

Bao nhiêu overlap là đủ? Mức thông thường là 10–20% chunk size. Với chunk 500 token, overlap 50–100 token. Nếu tài liệu có nhiều ranh giới câu mờ (ví dụ văn bản khoa học với các thuật ngữ nối tiếp), có thể tăng lên 25%. Luôn nhớ: overlap càng nhiều, số chunk càng tăng, chi phí lưu trữ và tìm kiếm càng lớn. Overlap quá cao (>50%) gây trùng lặp dữ liệu nghiêm trọng mà không mang lại lợi ích tương xứng.

Kiểm tra cấu hình: A/B Testing thực tế

Không có lý thuyết nào thay thế được việc chạy thử nghiệm trên dữ liệu thật. Quy trình A/B testing cho chunk size và overlap gồm các bước sau:

1. **Chuẩn bị bộ câu hỏi kiểm tra:** Khoảng 50–200 câu hỏi đại diện cho use-case thực tế, mỗi câu có kèm đáp án chính xác hoặc đoạn văn nguồn cần truy xuất.
2. **Tạo các cấu hình chunking:** Thay đổi chunk size (ví dụ 200, 400, 600, 800 token) và overlap (0%, 10%, 20%, 30%). Cố định chiến lược chunking (ví dụ recursive) và các tham số khác.
3. **Chạy retrieval và đo độ chính xác:** Với mỗi cấu hình, truy xuất top-k chunk (thường k=3 hoặc k=5) và tính *hit rate* – tỷ lệ câu hỏi mà chunk đúng nằm trong kết quả trả về. Đây là chỉ số quan trọng nhất ở giai đoạn chunking.
4. **Phân tích kết quả:** Chọn cấu hình có hit rate cao nhất. Nếu hai cấu hình bằng điểm, ưu tiên cấu hình có số chunk ít hơn (chi phí thấp hơn).
5. **Kiểm tra đầu ra LLM (tùy chọn):** Chạy một pipeline RAG đơn giản, so sánh chất lượng câu trả lời trên 20–30 câu mẫu. Đôi khi hit rate cao nhưng LLM vẫn trả lời kém do chunk chứa quá nhiều nhiễu – lúc đó cần giảm chunk size hoặc tăng k.

Công cụ hỗ trợ: Các framework như LangChain (với `TextSplitter` và `ChunkSizeOverlapEvaluator`) hoặc LlamaIndex (với `SentenceSplitter`) cho phép thay đổi tham số nhanh và tích hợp sẵn các metric đánh giá.

Ví dụ minh họa: Ảnh hưởng của chunk size đến retrieval

Giả sử bạn có một tài liệu gồm các mô tả ngắn về 10 sản phẩm, mỗi mô tả dài 1–2 câu (~50–100 token). Bộ câu hỏi kiểm tra gồm các câu như: “Sản phẩm A có tính năng gì nổi bật?” Độ chính xác kỳ vọng là 100% nếu chunk chứa đúng mô tả.

- **Chunk size = 50 token:** Mỗi chunk chỉ chứa một câu. Hit rate cao, vì câu hỏi thường trùng khớp chính xác với một chunk. Nhưng LLM nhận được rất ít ngữ cảnh, có thể trả lời thiếu thông tin.
- **Chunk size = 200 token:** Mỗi chunk chứa 2–3 sản phẩm. Nếu câu hỏi về một sản phẩm, retriever trả về cả chunk chứa sản phẩm đó cùng vài sản phẩm khác. Hit rate vẫn cao, nhưng LLM phải xử lý thông tin nhiễu. Với top k=1, chunk sai có thể chiếm ưu thế nếu câu hỏi không rõ ràng.

- **Chunk size = 500 token:** Mỗi chunk chứa gần như toàn bộ tài liệu. Hit rate gần như 100% vì chunk nào cũng chứa thông tin. Nhưng LLM nhận quá nhiều nhiễu; độ chính xác câu trả lời giảm rõ rệt và chi phí token tăng gấp 5–10 lần.

Thí nghiệm này cho thấy: chunk size quá nhỏ gây mất ngữ cảnh, quá lớn gây nhiễu; mức tối ưu là kích thước vừa đủ để mỗi chunk chứa một đơn vị thông tin trọn vẹn (một sản phẩm, một hàm API, một đoạn văn).

Checklist cấu hình chunking cho production

Trước khi đưa vào production, hãy kiểm tra từng mục sau:

- [ ] Xác định phân phối độ dài tài liệu: trung bình bao nhiêu token/câu, bao nhiêu câu/đoạn?
- [ ] Chọn chunk size khởi đầu dựa trên đơn vị ngữ nghĩa tự nhiên (một bước trong quy trình, một định nghĩa, một mô tả ngắn).
- [ ] Đặt overlap = 10–20% chunk size, kiểm tra với 5 câu hỏi ranh giới.
- [ ] Tạo bộ câu hỏi kiểm tra tối thiểu 50 câu, gắn nhãn chunk đúng.
- [ ] Chạy A/B testing với 3–4 mức chunk size và 2–3 mức overlap, đo hit rate.
- [ ] Chọn cấu hình có hit rate cao nhất và số chunk hợp lý (không quá nhiều).
- [ ] Kiểm tra thủ công 20 mẫu kết quả LLM để xác nhận chất lượng.
- [ ] Thiết lập monitoring: ghi log chunk size, overlap, số chunk truy xuất mỗi query vào production.
- [ ] Định kỳ (3–6 tháng) đánh giá lại khi dữ liệu thay đổi.

> ***Nguyên tắc vàng:*** *Chunk size không phải là hằng số; nó là hàm số của dữ liệu và câu hỏi. Luôn thử nghiệm trên dữ liệu thật trước khi chốt cấu hình production.*

Chunk size và overlap được tối ưu hóa tốt sẽ tạo nền tảng vững chắc cho bước tiếp theo: chuyển dữ liệu đã chunk vào Vector Database và thiết lập indexing. Chương tiếp theo sẽ hướng dẫn chi tiết cách chọn và cấu hình Vector DB để lưu trữ và truy vấn hiệu quả.

Chương 6: Vector Database: Lựa chọn và tích hợp

6.1 Kiến trúc và cách hoạt động của Vector DB

Kiến trúc của Vector Database

Khi bạn đã tối ưu chunk size và overlap, bước tiếp theo là lưu trữ các chunk đã được embedding thành vector vào một hệ thống có thể truy xuất nhanh. Cơ sở dữ liệu truyền thống (SQL, NoSQL) được thiết kế để tìm kiếm chính xác trên dữ liệu có cấu trúc hoặc dạng text, nhưng không thể xử lý hiệu quả phép toán "tìm vector gần nhất" trong không gian hàng trăm chiều. Với hàng triệu vector, quét toàn bộ (brute-force) tốn $O(n^2)$ thời gian — không khả thi cho ứng dụng real-time. Vector Database ra đời để giải quyết bài toán này bằng cách xây dựng chỉ mục (index) cho phép tìm kiếm lân cận gần đúng (Approximate Nearest Neighbor – ANN), giảm độ phức tạp xuống $O(\log n)$ hoặc $O(\sqrt{n})$ mà vẫn duy trì độ chính xác chấp nhận được.

Tại sao SQL/NoSQL không phù hợp?

Các database truyền thống lưu trữ dữ liệu dạng bảng hoặc document và hỗ trợ tìm kiếm bằng đối sánh chính xác (B-tree index) hoặc full-text search (inverted index). Với vector, không có phép so sánh "bằng nhau" — câu hỏi là "vector nào gần nhất?". Để trả lời, cần tính khoảng cách cosine hoặc Euclidean đến từng vector trong cơ sở dữ liệu. Một số NoSQL cho phép lưu trữ vector dưới dạng mảng số thực, nhưng thiếu cơ chế tối ưu hóa truy vấn lân cận: mỗi truy vấn phải quét toàn bộ tập dữ liệu. Với 1 triệu vector 768 chiều, mỗi truy vấn cần tính ~1.5 tỷ phép nhân — mất hàng giây trên CPU. Vector Database giải quyết vấn đề này bằng cách tổ chức lại dữ liệu thành cấu trúc chỉ mục đặc biệt.

Cơ chế ANN: Tốc độ đến từ sự xấp xỉ

ANN là họ thuật toán đánh đổi độ chính xác để đổi lấy tốc độ. Thay vì trả về k vector gần nhất chắc chắn, ANN trả về k vector xấp xỉ gần nhất với sai số cho phép. Trong thực tế, với RAG, sai số nhỏ (ví dụ thiếu 1-2% chunk phù hợp) ít ảnh hưởng đến chất lượng tổng thể, vì LLM có thể bù đắp nếu nhận đủ ngữ cảnh đúng. Hai phương pháp ANN phổ biến nhất trong Vector Database hiện nay là IVF (Inverted File Index) và HNSW (Hierarchical Navigable Small World).

IVF chia không gian vector thành các cụm (cluster) bằng thuật toán phân cụm K-means. Mỗi vector được gán vào cụm gần nhất. Khi truy vấn, chỉ tìm kiếm trong một số cụm gần tâm truy vấn nhất, giảm đáng kể số vector cần so sánh. IVF dễ triển khai, phù hợp

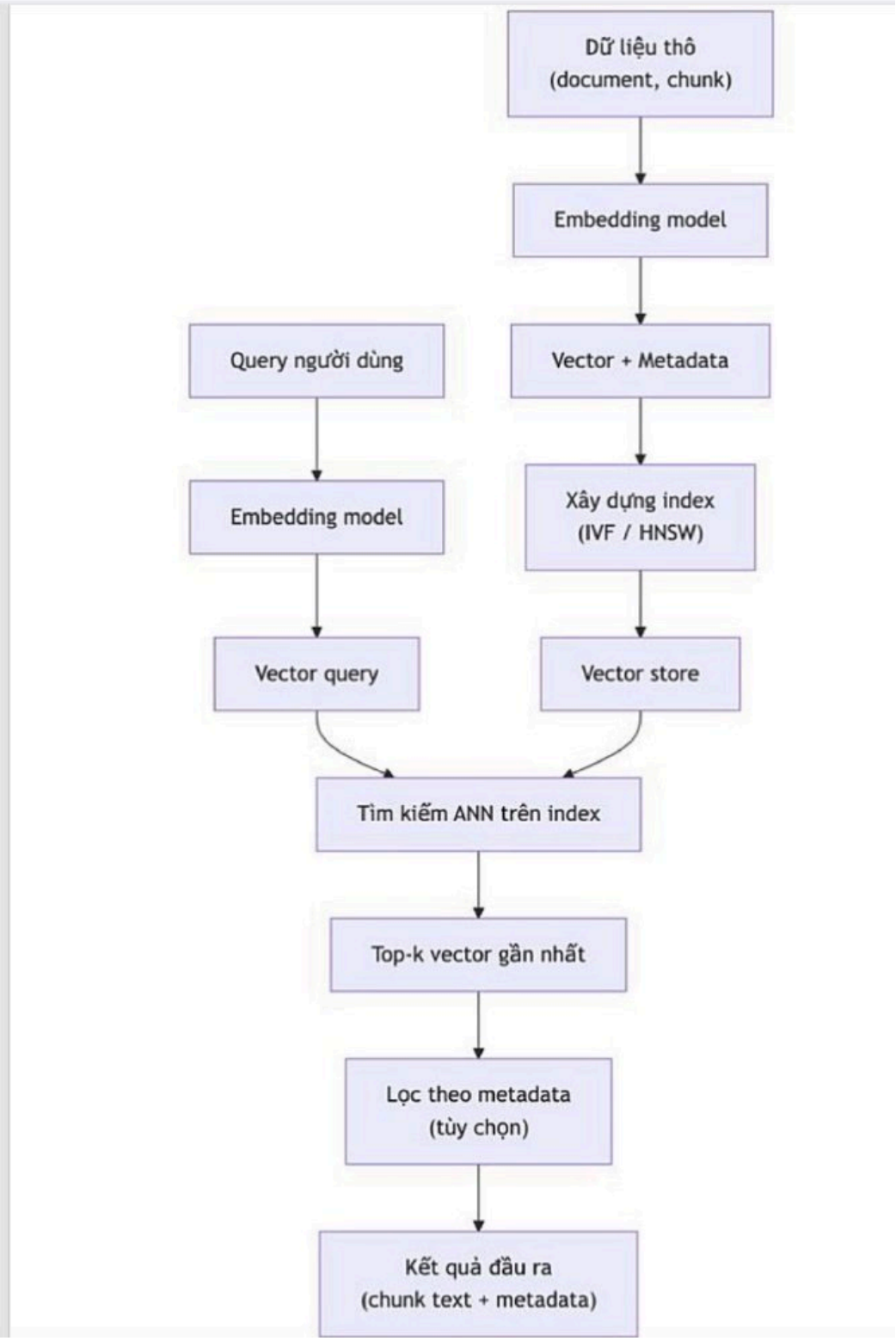
với dữ liệu tĩnh hoặc ít thay đổi. Nhược điểm là cần huấn luyện phân cụm trên toàn bộ dữ liệu, và chất lượng giảm nếu dữ liệu phân bố lệch.

HNSW xây dựng nhiều lớp đồ thị (layer) có cấu trúc phân cấp. Lớp trên cùng chứa các vector đại diện (xương), lớp dưới dày đặc hơn. Khi truy vấn, bắt đầu từ lớp trên, tìm kiếm local xuống dần, mỗi bước chỉ duyệt một phần nhỏ đồ thị. HNSW cho recall rất cao (thường >99% với k=10) và độ trễ thấp, phù hợp với dữ liệu động (thêm/xóa thường xuyên). Tuy nhiên, bộ nhớ sử dụng lớn hơn IVF (do lưu đồ thị), và việc xây dựng index chậm hơn.

Cả hai phương pháp đều hỗ trợ tham số điều chỉnh đánh đổi tốc độ – chính xác. Đối với RAG, HNSW thường là lựa chọn mặc định vì độ chính xác cao và khả năng xử lý dữ liệu cập nhật, trong khi IVF phù hợp khi bộ nhớ hạn chế hoặc dữ liệu lớn ít thay đổi.

Kiến trúc tổng quát của một Vector Database

Sơ đồ dưới đây minh họa các thành phần chính trong một Vector Database điển hình:


Dữ liệu thô
(document, chunk)
Embedding model
Vector + Metadata
Query người dùng
Embedding model
Xây dựng index
(IVF / HNSW)
Vector query
Vector store
Tìm kiếm ANN trên index
Top-k vector gần nhất
Lọc theo metadata
(tùy chọn)
Kết quả đầu ra
(chunk text + metadata)

Sơ đồ: Kiến trúc tổng quát Vector Database

Luồng dữ liệu từ trái sang phải:

1. **Ingestion:** Dữ liệu thô (chunk text) được đưa qua embedding model để tạo vector. Vector được ghép với metadata (ví dụ: document_id, nguồn, timestamp).
2. **Indexing:** Cặp vector–metadata được thêm vào index (HNSW hoặc IVF). Quá trình này có thể tốn thời gian tùy thuộc vào kích thước dữ liệu và cấu hình index.
3. **Query:** Query người dùng được embedding thành vector truy vấn. Vector này đi vào index để tìm kiếm ANN, trả về top-k vector gần nhất.
4. **Metadata filtering:** Trước hoặc sau tìm kiếm ANN, có thể áp dụng bộ lọc metadata (ví dụ: chỉ lấy document từ năm 2024). Bước này ảnh hưởng đến hiệu năng và sẽ được phân tích chi tiết trong phần sau.
5. **Response:** Kết quả là danh sách các chunk gốc kèm metadata, sẵn sàng cho bước reranking hoặc generation.

Ví dụ cụ thể về dữ liệu vector và metadata

Giả sử bạn có một tài liệu kỹ thuật chia thành 3 chunk:

- **Chunk 1:** "Hệ thống RAG sử dụng Vector Database để lưu trữ embedding." → Embedding vector 768 chiều (`v1`), metadata: `{doc\_id: "doc\_001", chunk\_id: 1, source: "rag\_doc.pdf"}` .
- **Chunk 2:** "ANN là thuật toán tìm kiếm xấp xỉ, đánh đổi độ chính xác lấy tốc độ." → Vector `v2` , metadata: `{doc\_id: "doc\_001", chunk\_id: 2}` .
- **Chunk 3:** "HNSW thường cho recall cao hơn IVF." → Vector `v3` , metadata: `{doc\_id: "doc\_001", chunk\_id: 3}` .

Khi người dùng hỏi: "ANN khác gì brute-force?", query được embedding thành vector `q` . Vector DB sử dụng index HNSW để trả về top-2 chunk gần nhất với `q` . Giả sử kết quả là `v2` và `v3` . Hệ thống đọc metadata và trả về đoạn text tương ứng cho LLM.

Metadata Filtering: Lọc trước hay lọc sau?

Một câu hỏi quan trọng trong thiết kế Vector DB là thứ tự áp dụng bộ lọc metadata. Có hai chiến lược:

- **Pre-filtering:** Lọc metadata trước, chỉ tìm kiếm ANN trên tập con dữ liệu thỏa mãn điều kiện. Cách này giảm số lượng vector cần xét, nhưng có thể bỏ sót kết quả nếu tập con quá nhỏ hoặc bộ lọc quá hẹp.
- **Post-filtering:** Tìm kiếm ANN trên toàn bộ index, sau đó loại bỏ các kết quả không thỏa mãn metadata. Cách này đảm bảo không mất kết quả tiềm năng, nhưng có thể trả về ít hơn k vector mong muốn (nếu nhiều kết quả bị lọc) và lãng phí tính toán trên các vector không phù hợp.

Phần lớn Vector DB (Pinecone, Weaviate, Qdrant) hỗ trợ cả hai cơ chế, với tùy chọn `pre-filter` hoặc `post-filter`. Lựa chọn phụ thuộc vào tính chất lọc: nếu bộ lọc chọn lọc cao (ví dụ: chỉ 1% dữ liệu), pre-filter hiệu quả hơn; nếu bộ lọc khoan dung (ví dụ: 80% dữ liệu), post-filter đơn giản và đủ nhanh.

Lưu ý về hiệu năng index

Index không phải là giải pháp vạn năng. Một số yếu tố ảnh hưởng đến tốc độ và độ chính xác:

- **Số chiều vector:** Càng cao, tính toán khoảng cách càng nặng. Các model embedding phổ biến (text-embedding-ada-002, all-MiniLM-L6-v2) tạo vector 384–1536 chiều. Chọn model có số chiều vừa phải giúp tăng tốc.
- **Kích thước index:** Với hàng chục triệu vector, index HNSW có thể ngốn hàng chục GB RAM. Một số DB hỗ trợ lưu index trên đĩa (disk-based) nhưng độ trễ cao hơn.
- **Tham số index:** HNSW có tham số `ef\_construction` (độ chính xác khi xây dựng) và `M` (số kết nối tối đa mỗi node). Tăng hai tham số này cải thiện recall nhưng tăng thời gian xây dựng và bộ nhớ. Với RAG, thông thường chọn M=16–32, ef_construction=200–500, và `ef` (thời gian tìm kiếm) từ 100–200.

> ***Nguyên tắc:*** *Luôn benchmark trên dữ liệu thật để chọn tham số index phù hợp. Không có cấu hình chung cho mọi trường hợp.*

Hiểu rõ kiến trúc Vector Database giúp bạn đưa ra quyết định chính xác khi chọn giải pháp và cấu hình cho hệ thống RAG. Phần tiếp theo sẽ so sánh ba Vector Database phổ biến — FAISS, Pinecone và Weaviate — dựa trên các yếu tố như chi phí, độ trễ, và tính năng để bạn có thể chọn công cụ phù hợp nhất.

6.2 So sánh FAISS, Pinecone và Weaviate cho Software Project

So sánh thực tế: FAISS, Pinecone và Weaviate

Sau khi hiểu kiến trúc tổng quan của Vector Database, bước tiếp theo là chọn công cụ cụ thể. Ba lựa chọn phổ biến nhất hiện nay — FAISS, Pinecone và Weaviate — đại diện cho ba hướng tiếp cận khác nhau: thư viện thuần (library), dịch vụ được quản lý (managed service) và cơ sở dữ liệu mã nguồn mở (open-source DB). Mỗi công cụ có trade-off riêng về khả năng mở rộng, độ trễ, chi phí và độ phức tạp vận hành. Phần này sẽ phân tích chi tiết để giúp bạn đưa ra quyết định phù hợp với dự án RAG của mình.

FAISS: Thư viện vector hiệu năng cao, tự chủ hoàn toàn

FAISS (Facebook AI Similarity Search) là một thư viện mã nguồn mở do Meta phát triển, chuyên dùng để tìm kiếm vector và clustering. FAISS không phải là một cơ sở dữ liệu hoàn chỉnh — nó chỉ là một thư viện tính toán tập trung vào việc xây dựng index và thực thi ANN search. Điều này có nghĩa là bạn phải tự quản lý toàn bộ pipeline lưu trữ, cập nhật index, và triển khai dưới dạng service nội bộ.

Ưu điểm nổi bật:

- **Hiệu năng cao:** FAISS hỗ trợ nhiều loại index (HNSW, IVF, PQ) với khả năng tinh chỉnh mịn, cho phép đạt recall gần 100% trên dữ liệu lớn nếu cấu hình đúng.
- **Không phụ thuộc vendor:** Bạn kiểm soát hoàn toàn việc lưu trữ và vận hành, không bị ràng buộc bởi pricing model hay giới hạn SLA của bên thứ ba.
- **Chi phí thấp ở quy mô nhỏ:** Với dưới 1 triệu vector, bạn có thể chạy FAISS trên một instance CPU đơn giản, chi phí chỉ bao gồm hạ tầng.

Nhược điểm chính:

- **Vận hành phức tạp:** Bạn phải tự thiết lập server, triển khai API, scaling, backup, monitoring. Không có sẵn tính năng CRUD, metadata filtering, hay replication.
- **Khó scale ngang:** Khi vượt quá một server, bạn cần tự triển khai sharding và load balancing, đòi hỏi kiến thức hệ thống phân tán.
- **Không có tính năng DB tích hợp:** Không hỗ trợ lưu trữ bền vững (persistence) mặc định; phải tự code lưu vào disk, kết hợp với key-value store để quản lý metadata.

Khi nào nên dùng FAISS?

- Bạn có đội ngũ DevOps mạnh, sẵn sàng đầu tư vào hạ tầng tự quản.
- Dữ liệu vector ổn định (ít cập nhật), scale dưới 10 triệu vector.
- Yêu cầu latency cực thấp (<10ms) và bạn muốn tối ưu đến từng chi tiết cấu hình index.

Pinecone: Managed service đơn giản, chi phí cao

Pinecone là dịch vụ Vector Database được quản lý hoàn toàn, cung cấp API RESTful đơn giản để lưu trữ và truy vấn vector. Người dùng không cần quan tâm đến hạ tầng bên dưới — Pinecone tự động xử lý index, scaling, replication và monitoring.

Ưu điểm nổi bật:

- **Dễ triển khai:** Chỉ cần tạo index qua dashboard hoặc SDK, upload vector và bắt đầu query. Không cần cấu hình server hay index tuning.
- **Tự động scale:** Pinecone tự động mở rộng tài nguyên khi dữ liệu tăng, đảm bảo latency ổn định mà không cần can thiệp thủ công.
- **Tích hợp sẵn metadata filtering:** Hỗ trợ pre-filter và post-filter, cùng với joins qua namespace và metadata.

Nhược điểm chính:

- **Chi phí cao:** Pinecone tính phí dựa trên số lượng vector và dung lượng lưu trữ. Với 10

- triệu vector 768 chiều, chi phí có thể lên tới 500-1000/tháng, gấp 5-10 lần so với tự host FAISS trên cloud.
- **Vendor lock-in:** Dữ liệu và index nằm trên hạ tầng của Pinecone; việc di chuyển sang nền tảng khác mất công export và tái tạo index.
- **Ít kiểm soát:** Không thể tinh chỉnh cấu hình index (HNSW parameters) hay cache layer; phải chấp nhận cấu hình mặc định.

Khi nào nên dùng Pinecone?

- Bạn muốn tập trung vào product, không muốn quản lý hạ tầng.
- Dữ liệu thay đổi liên tục, cần tự động scaling và zero-ops.
- Ngân sách cho phép, và bạn chấp nhận trả premium cho sự tiện lợi.

Weaviate: Open-source DB với hybrid search tích hợp

Weaviate là một cơ sở dữ liệu mã nguồn mở được thiết kế riêng cho vector search, nhưng cung cấp đầy đủ tính năng của một hệ quản trị cơ sở dữ liệu quan hệ. Nó hỗ trợ lưu trữ dữ liệu có cấu trúc (object, property) song song với vector, và tích hợp sẵn cả ANN search lẫn keyword search (BM25).

Ưu điểm nổi bật:

- **Hybrid search:** Kết hợp vector và keyword trong một query duy nhất, với khả năng điều chỉnh trọng số. Đây là lợi thế lớn so với FAISS (chỉ vector) và Pinecone (hỗ trợ keyword mức hạn chế).
- **Cơ sở dữ liệu hoàn chỉnh:** Lưu trữ object với schema, hỗ trợ CRUD, transactional write, và backup. Không cần tích hợp thêm storage khác.
- **Mã nguồn mở, tự do:** Có thể tự host (on-premise hoặc cloud) hoặc dùng bản managed service (Weaviate Cloud). Không bị vendor lock-in nếu tự host.

Nhược điểm chính:

- **Độ phức tạp vận hành:** Mặc dù đơn giản hơn FAISS, nhưng Weaviate vẫn yêu cầu kiến thức về Docker, Kubernetes, và cấu hình index. Scaling lên hàng chục triệu vector đòi hỏi planning kỹ càng.
- **Hiệu năng ANN thấp hơn FAISS:** Do phải phục vụ nhiều tính năng khác nhau, latency ANN thường cao hơn FAISS chuyên dụng trong cùng cấu hình.
- **Cộng đồng nhỏ hơn:** So với FAISS (10K+ stars) hay Pinecone (hệ sinh thái mạnh), Weaviate có cộng đồng nhỏ hơn, tài liệu và troubleshoot ít hơn.

Khi nào nên dùng Weaviate?

- Bạn cần hybrid search (vector + keyword) ngay trong cùng một cơ sở dữ liệu.
- Dữ liệu có cấu trúc phức tạp, cần lưu trữ object và query kết hợp nhiều điều kiện.
- Bạn muốn mã nguồn mở nhưng không muốn tự lắp ghép như FAISS, chấp nhận trade-off về hiệu năng.

Bảng so sánh tổng quan

Bảng: So sánh FAISS, Pinecone và Weaviate

Tiêu chí	FAISS	Pinecone	Weaviate
Loại	Thư viện (Library)	Managed Service	Open-source DB
Cài đặt	Tự cài qua pip, không cần hạ tầng DB	Dùng SDK, không cài server	Docker / Kubernetes / Managed
Scalability	Thủ công (phân chia index, sharding)	Tự động mở rộng	Tự động (Kubernetes) nhưng cần cấu hình
Độ trễ (p99)	<5ms (in-memory)	10-20ms (phổ biến)	15-30ms (phụ thuộc cấu hình)
Chi phí (10M vector)	50-150/tháng (tự host)	500-1000/tháng	100-300/tháng (tự host)
Hybrid search	Không	Có (giới hạn)	Có (mạnh mẽ)
Metadata filtering	Cần tự code	Có sẵn	Có sẵn
Backup & Replication	Tự làm	Tự động	Cấu hình
Độ phức tạp vận hành	Cao	Thấp	Trung bình
Vendor lock-in	Không	Có	Không (nếu tự host)
Phù hợp với	PoC, đội mạnh infra	Đội thiếu infra, product nhanh	Hybrid search, đội trung bình

Checklist lựa chọn Vector Database cho dự án RAG

Trước khi chọn, hãy trả lời các câu hỏi sau:

1. Quy mô dữ liệu là bao nhiêu?

- Dưới 1M vector → FAISS (đơn giản, free) hoặc Weaviate (nếu cần hybrid).
- 1M-10M vector → Weaviate (self-host) hoặc Pinecone (nếu tiền).
- Trên 10M vector → Pinecone hoặc FAISS + custom scaling.

2. Tần suất cập nhật dữ liệu?

- Rất ít cập nhật → FAISS (rebuild index định kỳ).
- Cập nhật liên tục → Pinecone hoặc Weaviate (hỗ trợ incremental index).

3. **Yêu cầu về latency?**

- <10ms → FAISS (in-memory).
- 10-50ms → Pinecone hoặc Weaviate.

4. **Có cần hybrid search không?**

- Không → FAISS hoặc Pinecone.
- Có → Weaviate (mạnh nhất) hoặc tự triển khai Elasticsearch + FAISS.

5. **Đội ngũ vận hành?**

- Có DevOps mạnh → FAISS hoặc Weaviate.
- Không có đội infra → Pinecone.

6. **Ngân sách?**

- Hạn chế → FAISS hoặc Weaviate tự host.
- Dư giả → Pinecone (tiết kiệm thời gian).

Ví dụ kịch bản sử dụng thực tế

Kịch bản A: Chatbot hỗ trợ kỹ thuật nội bộ (100K tài liệu, scale nhỏ)

- Lựa chọn: **FAISS**.
- Lý do: Dữ liệu ít, không yêu cầu cập nhật real-time, đội ngũ có thể Docker hóa FAISS + FastAPI trong vài ngày. Chi phí hạ tầng ~20/tháng. **Kịch bản B: Search engine cho sàn thương mại (5 triệu sản phẩm, cần hybrid search)** - Lựa chọn: **Weaviate**. - Lý do: Cần kết hợp vector tìm kiếm ngữ nghĩa và keyword tìm chính xác (SKU). Weaviate hỗ trợ hybrid search native, giảm độ phức tạp tích hợp. **Kịch bản C: Ứng dụng SaaS phục vụ khách hàng (50 triệu vector, không có đội ngũ infra)** - Lựa chọn: **Pinecone**. - Lý do: Team tập trung vào product, không muốn quản lý server. Pinecone tự động scale, đảm bảo SLA 99.9%. Chi phí ~800/tháng chấp nhận được so với nhân sự thuê.

Việc chọn đúng Vector Database không có đáp án duy nhất; nó phụ thuộc vào trade-off giữa chi phí, độ phức tạp và yêu cầu hiệu năng. Trong chương tiếp theo, chúng ta sẽ xây dựng pipeline indexing hoàn chỉnh, bất kể bạn chọn công cụ nào.

Chương 7: Xây dựng Pipeline Indexing tự động

7.1 Thiết kế luồng ETL cho dữ liệu Vector

Xây dựng Indexing Pipeline

Hãy tưởng tượng bạn là kỹ sư của một công ty luật. Mỗi ngày, hệ thống nhận hàng trăm hợp đồng PDF mới, và bạn phải cập nhật chúng vào cơ sở dữ liệu vector để chatbot hỗ trợ giải đáp thắc mắc. Làm thủ công từng bước – đọc file, copy text, chạy embedding, insert vào DB – vừa mất thời gian vừa dễ sai sót. Một tệp định dạng lỗi, một kết nối mạng đứt, và toàn bộ tiến trình đổ bể. Đây chính là lúc bạn cần một **indexing pipeline** tự động hóa toàn bộ quy trình Extract, Transform, Load (ETL) chuyên biệt cho dữ liệu vector.

Tổng quan pipeline ETL cho RAG

Indexing pipeline là một hệ thống tự động hóa các bước: lấy dữ liệu thô (Extract), chuyển đổi thành vector qua chunking và embedding (Transform), và đẩy vào Vector Database (Load). Mỗi bước đều có những yêu cầu kỹ thuật riêng, và sự đồng bộ giữa chúng quyết định chất lượng của toàn bộ hệ thống RAG.